
Software Requirements Specification

for

Community Social Network Website LKForum

Release 1.0

Version 1.0 approved

Prepared by Group 4

Process Impact

October, 2025

Table of Contents

1. Introduction	1
1.1 Purpose	1
1.2 Document Conventions	1
1.3 Project Scope and Product Features	1
1.4 References	1
2. Overall Description	1
2.1 Product Perspective	1
2.2 User Classes and Characteristics	2
2.3 Operating Environment	2
2.4 Design and Implementation Constraints	2
2.5 Assumptions and Dependencies	2
3. System Features	3
3.1 User Account and Profile Management	3
3.1.1 Description	3
3.1.2 Functional Requirements	3
3.2 Create and Manage Communities	4
3.2.1 Description	4
3.2.2 Functional Requirements	4
3.3 Create and manage post content	4
3.3.1 Description	4
3.3.2 Functional Requirements	5
3.4 Interaction with post and community system	5
3.4.1 Description	5
3.4.2 Functional Requirements	5
3.5 Platform Administration	6
1.1.1 Description	6
1.1.2 Functional Requirements	6
2. Data Requirements	8
2.1 Logical Data Model	8
2.2 Data Dictionary	8
2.3 Reports	11
2.4 Data Integrity, Retention, and Disposal	13
3. External Interface Requirements	14
3.1 User Interfaces	14
3.2 Software Interfaces	14
3.3 Hardware Interfaces	16
3.4 Communications Interfaces	16

1. Introduction

1.1 Purpose

This SRS describes the functional and nonfunctional requirements for software release 1.0 of the Community Social Network Website (LKForum). This document is intended to be used by the members of the project team who will implement and verify the correct functioning of the system. Unless otherwise noted, all requirements specified here are committed for release 1.0.

1.2 Document Conventions

No special typographical conventions are used in this SRS.

1.3 Project Scope and Product Features

LKForum will enable users to create and join topic-based online communities. Users can post content, leave comments, and engage with both articles and other users. The system's objective is to build a vibrant and organized environment for discussion.

1.4 References

2. Overall Description

2.1 Product Perspective

Community Social Network LKForum is a new website that provides a comprehensive, modern solution for building and managing online communities. The context diagram in Figure 1 illustrates the external entities and system interfaces for release 1.0. The system is expected to evolve over several releases, ultimately connecting to third-party social media APIs for authentication (Google, Facebook) and expanding to include advanced content monetization tools and mobile-native applications.

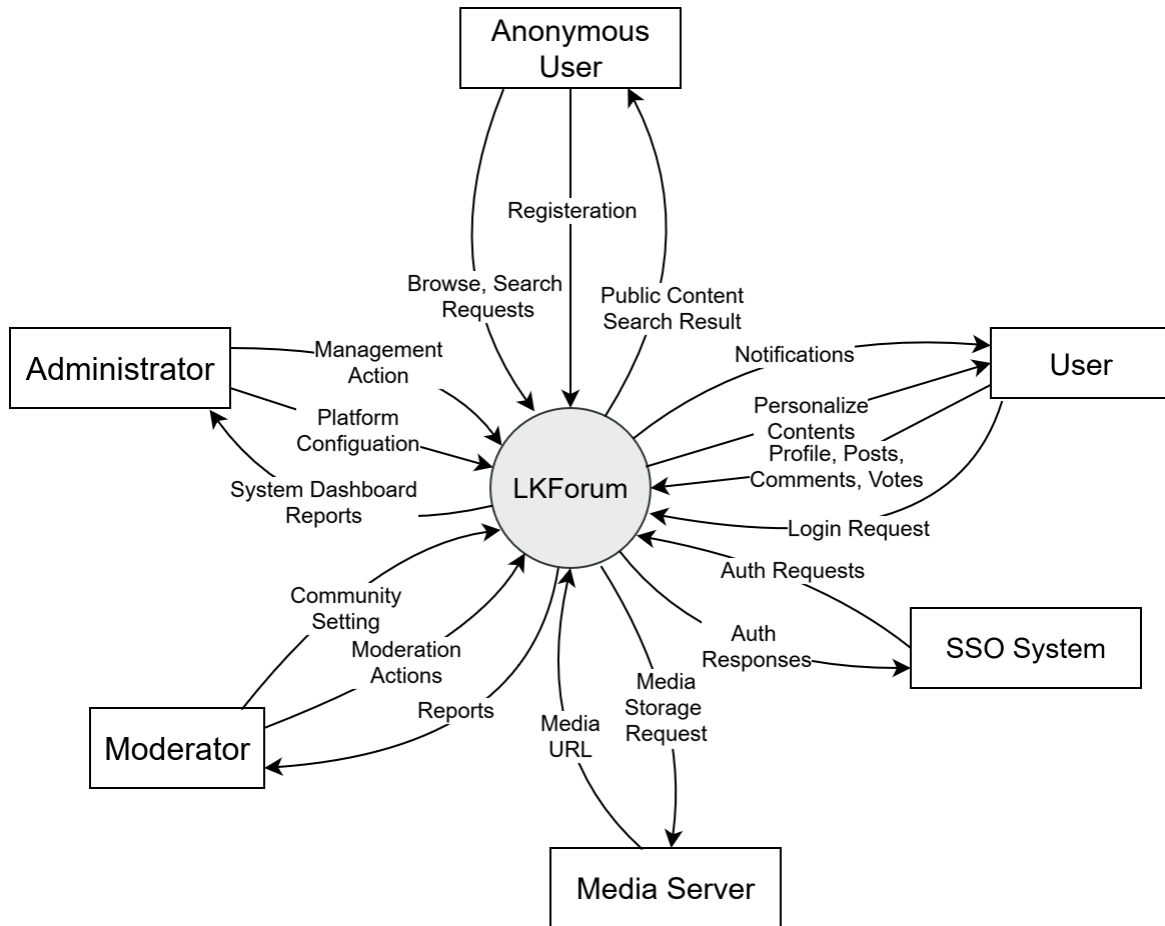


Figure 1. Context diagram for release 1.0 of LKForum.

2.2 User Classes and Characteristics

User	A registered member of the platform. They can participate in communities, create posts, comment, vote on posts and comments, customize their personal profile, and follow other users. This is the largest user group.
Guest User	A user who has not logged in. They mainly interact with the system by viewing public content and performing searches.
Community Moderator	A member granted management rights over one or more specific communities. They can approve or delete posts and comments, ban or suspend members who violate rules, pin important posts, and customize community rules and appearance.
System Administrator	The highest authority on the entire platform. They are responsible for managing all communities (create, delete), handling serious violation reports, managing user accounts system-wide, and ensuring stable system operation.

2.3 Operating Environment

- OE-1: LKForum must operate correctly on modern web browsers including: Google Chrome (latest version), Mozilla Firefox (latest version), Apple Safari (latest version), and Microsoft Edge (latest version).
- OE-2: Users can access LKForum via the Internet from desktop computers, laptops, and tablets. The interface must have responsive design to be compatible with various screen sizes.

2.4 Design and Implementation Constraints

- CO-1: The user interface (front-end) source code must be developed using the Svelte framework.
- CO-2: The system must use a standard MongoDB NoSQL database with a back-end developed using the Go framework.
- CO-3: All HTML code must adhere to the HTML 5.0 standard.

2.5 Assumptions and Dependencies

- AS-1: It is assumed that users have a stable Internet connection to access and use the platform's features.
- DE-1: The social media login feature (Google, Facebook) depends on the APIs and policies of these third-party service providers. Any changes on their part may affect the operation of this feature.

3. System Features

3.1 User Account and Profile Management

3.1.1 Description

Each user has an account and personal profile to manage information, track activity, and customize privacy settings. Priority: High.

3.1.2 Functional Requirements

User. Auth: User Authentication	
.Register:	The system will allow new users to register an account.
.Email:	Provide an email address, username, and password to register a new account.
.SSO:	The system supports registration using third-party accounts such as Google or Facebook.
.Validation:	The system will validate that the registration email is unique and the password meets minimum security criteria. .Login: The system will allow users to log into an existing account using email and password or through a linked SSO.
.OK:	If the login information is correct, the user will be provided with a new session and returned to the homepage with the logged-in account.

.NEG:	If the login information is incorrect, the system will display an error message and guide the user on how to recover their password if needed. .Anonymous: The system will allow users to view content without logging in (anonymous mode), but will require login to perform actions such as posting, commenting, or voting.
.Logout:	The system will provide functionality to allow users to log out of the current session.
User.Profile: Personal Profile Management	
.View:	When accessing a profile, the system will display public information including: Avatar (profile picture); Description.
.Edit:	Users can edit their own profile. The system will allow users to: Upload and change their avatar; Edit the description content.
.Save:	All changes to the profile must be explicitly confirmed and saved by the user.
.Delete:	Users have the ability to delete their account and all related information, must re-confirm their password.
User.History: Activity History	
.Access:	On their profile page, users can access sections to review their activity history.
.Posts:	The system will display a list of all posts the user has created, sorted chronologically.
.Comments:	The system will display a list of all comments the user has written, sorted chronologically.
.Votes:	The system allows users to review the history of posts and comments they have voted on (this feature is only visible to the user themselves).
User.Privacy: Privacy Settings	
.Access:	The system provides a dedicated settings page for users to manage their privacy options.
.Profile Visibility:	Users can set their profile visibility to "Public" (everyone can see) or "Private" (only followers can see).
.Activity Privacy:	Users can choose to hide certain activities, such as their voting history, from their public profile.
User.Social: Social Interaction	
.Follow:	The system allows a user to follow another user from their profile page.
.Unfollow:	The system allows a user to unfollow a user they are currently following.
.Display:	The profile page will display two lists: the number of users they are following (following) and the number of users following them (followers).
.Notification	User will receive a notification about selected activity of the followed user.

3.2 Create and Manage Communities

3.2.1 Description

Any logged-in user can create a new community. The community creator (Owner) can customize the appearance and appoint others as Moderators. Priority = High.

3.2.2 Functional Requirements

Community.Create: Create new community	
.Name:	When creating a community, the user must provide a unique identifier name (e.g., c/community-name). The system will check the uniqueness of this name.
.Details:	Users must provide a public title and a short description for the community.
.Create:	After successful creation, the user will automatically be assigned the "Owner" role for that community.
Community.Manage: Manage community settings	
.Customize:	Owner and Mod can customize the community's appearance, including: banner, logo, and color scheme (theme).
.Rules:	Owner and Mod can create, edit, and delete specific rules (guidelines) for their community. These rules will be displayed publicly.
.Roles:	Owner can invite other users to be Moderators (Mod) or Contributors. Owner can also remove these roles.
.PinPost:	Owner and Mod can pin one or more important posts to the top of the community page.
.Announce:	Owner and Mod can create special announcement posts.

3.3 Create and manage post content

3.3.1 Description

A logged-in user (Member) can create new posts in a community, and edit or delete their own posts. Members can also save posts as drafts. **Priority: High.**

3.3.2 Functional Requirements

Post.Create: Create new post	
.Select	
Community:	When creating a post, LKForum will require the user to select a community to post in.
.SelectType:	LKForum will allow users to select the content type of the post, including: Text, Photo, Album, Video, Link, GIF, and Poll.
.Input:	The user will enter the title and content (or link/file) corresponding to the selected post type.
.Submit:	When the user submits, the system will validate the content and publish the post to the selected community with public visibility.
.Failure:	If the post violates automated rules (e.g., spam), LKForum will reject the post and notify the user.
Post.Manage: Edit, Delete, and Save Draft	
.Draft:	During composition, the system will provide a "Save Draft" option. The post will be saved to the user's personal profile and will not be publicly visible.
.AccessDraft:	During composition, the system will provide a "Save Draft" option. The post will be saved to the user's personal profile and will not be publicly visible.

.Edit:	The system will allow users to edit the content and title of posts they have created.
.History:	After editing, the system will save the change history and display an "edited" indicator on the post.
.Delete:	The system will allow users to delete posts they have created. A deleted post will no longer be publicly visible.

Post.Share: Share post

.CrossPost:	The system will provide a "Cross-post" option on an existing post.
.SelectTarget:	The user can select another community they are a member of to repost the article.
.Link:	The new post (cross-post) will display an embedded link pointing to the original post.

3.4 Interaction with post and community system

3.4.1 Description

A logged-in user (Member) can interact with content by voting on posts and comments, as well as by adding new comments and replying to create discussion threads. **Priority: High.**

3.4.2 Functional Requirements

Interact.Vote: Vote on posts and comments

.PostVote:	The system will allow users to vote on posts..Comment
.CommentVote:	The system will allow users to "upvote" or "downvote" comments.
.NoSelfVote:	Users are not allowed to vote on their own posts or comments.
.Display:	The total vote score (upvotes minus downvotes) will be displayed publicly next to the post or comment.

Interact.Comment: Comment and Reply

.Add:	The system will allow users to add new comments to a post.
.Reply:	The system will allow users to reply to an existing comment.
.Thread:	All comments and replies will be displayed in a threaded structure.\
.Collapse:	The system will provide functionality for users to collapse or expand a comment thread.
.Delete:	Users can delete comments they have written.
.Tag:	The system will allow users to tag another user in a comment using the @username syntax.

3.5 Platform Administration

1.1.1 Description

The System Administrator has the highest access rights to monitor and manage the entire platform, ensuring compliance with general terms of service. **Priority: Medium.**

1.1.2 Functional Requirements

Admin.User: User Management

- .List: The system will provide Admin with an interface to view and search the entire list of users on the system.
- .ViewProfile: Admin can view the profile details of any user, including private information (e.g., registration email).
- .Action: Admin has the right to suspend or permanently delete a user account that seriously violates the general terms of service.
- .Reports: Admin will review and handle community reports related to a specific user's behavior.

Admin.Community: Community Management

- .List: The system will provide Admin with an interface to view and search the entire list of active communities.
- .Review: Admin can review reports related to a community (e.g., abandoned community, contains spam, or violates rules).
- .Action: Based on the review, Admin has the right to ban a community (switch to read-only mode) or permanently delete it from the platform.

Admin.Post: Post Management

- . Remove: Admin has the right to remove any post or comment on the entire platform without the permission of community Mods.
- . Reports: Admin will review and handle content reports (posts/comments) that have been complained about or escalated from the community level.
- .Search: Admin can search for content (posts/comments) system-wide based on keywords or ID.

Admin.Stats: Platform Statistics and Reports

- .Dashboard: The system will provide a visual overview dashboard for Admin.
- .UserStats: The dashboard must display: total registered users, and number of new registered users (by day/week/month).
- .ActivityStats: The dashboard must display: total number of posts and total number of comments created platform-wide.
- .PostStats: The dashboard must display: number of daily active users (DAU) and number of monthly active users (MAU).

2. Data Requirements

2.1 Logical Data Model

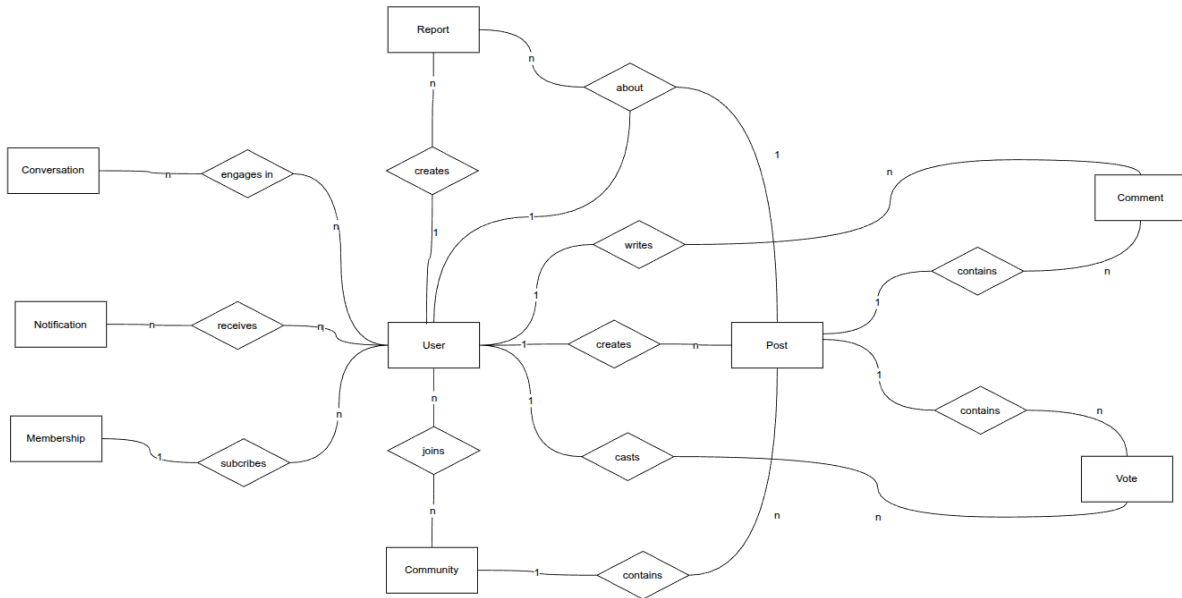


Figure 2. Partial data model for release 1.0 of the Cafeteria Ordering System.

2.2 Data Dictionary

Data Element	Description	Composition or Data Type	Length	Values
author id	Unique identifier of the user who created a piece of content (e.g., a post or comment).	objectId	24	Must correspond to an existing user id.
community id	Unique identifier for a specific community or sub-forum.	objectId	24	System-generated, unique.
community name	The public name of a community.	alphanumeric	50	Must be unique.

Data Element	Description	Composition or Data Type	Length	Values
content	The main substance of a message, post, or comment.	text		
conversation id	Unique identifier for a private message thread between users.	objectId	24	System-generated, unique.
created at	The exact date and time when an item (e.g., user account, post, comment) was created.	timestamp		System-generated.
description	A text field used to provide details about an item, such as a community or a report.	text		
email	A user's email address, used for login and receiving notifications.	alphanumeric	100	Must be a valid email format; unique.
is deleted	A status indicating whether an item has been marked for deletion.	boolean		true or false.
is private	A setting for a community to determine if it is visible to the public.	boolean		true or false.

Data Element	Description	Composition or Data Type	Length	Values
members	The list of users who are part of a private conversation.	array of user id		
members count	The total number of users who have joined a community.	integer		
message	The text content of a notification sent to a user.	text		
password	A secret string of characters used by a user to log in to their account.	string	255	Stored in a secure, hashed format.
post content	The main body of a post, which could contain text, a poll, or a video.	text + poll + video		
post id	Unique identifier for a single post within the forum.	objectId	24	System-generated, unique.
post status	The current state of a post.	alphabetic	16	"published", "pending approval", "deleted".
post type	The category or format of a post.	alphabetic	20	"text", "poll", "video".
reason	The explanation provided when a user submits a report.	alphabetic	50	
role	The permission level assigned to a user	alphabetic	16	"user", "moderator", "admin".

Data Element	Description	Composition or Data Type	Length	Values
	within a community or the system.			
sender id	The unique identifier of the user who sent a private message.	objectId	24	Must correspond to an existing user id.
target id	The unique identifier of an item that is being acted upon (e.g., reported or voted on).	objectId	24	
target type	The type of item that is being acted upon.	alphanumeric	16	"post", "comment".
user id	Unique identifier that the system assigns to each user account.	objectId	24	System-generated, unique.
user name	The public name a user chooses to be identified by on the forum.	alphanumeric	50	Must be unique.
vote value	The numerical representation of a user's vote.	integer		1 for an upvote, -1 for a downvote.
votes count	The summary of upvotes and downvotes for a post or comment.	up count + down count		

2.3 Reports

4.3.1 My Activity History Report

Report ID:	LKF-RPT-1
Report Title:	My Activity History
Report Purpose:	A user wants to see a list of all posts and comments they have created over a specified time period, so they can easily find and review their past contributions.
Priority:	Medium
Report Users:	Regular User
Data Sources:	Database of user's posts and comments.
Frequency and Disposition:	Report is generated on demand by the user. Displayed on the user's web browser.
Latency:	Complete report must be displayed within 3 seconds after it is requested.
Report Body:	Fields shown: - Content Type ("Post" or "Comment") - Post Title / Comment Snippet - Community Name - Upvotes - Creation Date Selection Criteria: Date range specified by the user. Sort Criteria: Reverse chronological order (newest first).
Interactivity:	User can click on any item in the report to navigate directly to that post or comment.
Security Access Restrictions:	A user may only retrieve their own activity history.

4.3.2 Community Engagement Report

Report ID:	LKF-RPT-2
Report Title:	Community Engagement Report
Report Purpose:	A Community Moderator or System Admin wants to see a list of the most engaging posts within a specific community over a time period, so they can understand content trends and identify popular topics.
Priority:	High

Report ID:	LKF-RPT-2
Report Users:	Moderator, Admin
Data Sources:	Database of posts, votes, and comments.
Frequency and Disposition:	Report is generated on demand. Displayed on a private admin/moderation dashboard.
Latency:	Complete report must be displayed within 5 seconds after it is requested.
Report Body:	Fields shown: - Post Title - Author - Net Score (Upvotes - Downvotes) - Total Comments - Creation Date Selection Criteria: Date range (e.g., "This Week", "This Month") and Community Name. Sort Criteria: By Net Score (descending) or by Total Comments (descending).
Interactivity:	Moderator can click on a post to view it, or click on the author's name to see their profile.
Security Access Restrictions:	Moderators can only generate reports for communities they manage. Admins can generate reports for any community.

2.4 Data Integrity, Retention, and Disposal

DI-1: The system shall retain user-deleted content (such as posts and comments) in a deactivated state for 30 days. After this period, the content will be permanently purged from the system. This allows users a grace period to recover accidentally deleted items.

DI-2: When a user requests to delete their account, the account shall be immediately deactivated and scheduled for permanent deletion. All personally identifiable information and associated content will be permanently deleted or anonymized after 90 days, unless required to be kept for legal reasons.

DI-3: Moderation data, including content reports submitted by users and action logs taken by moderators, shall be retained for one year after the report is closed to monitor for patterns of abuse.

DI-4: Core content such as active posts and comments will be retained indefinitely as long as the author's account and the community they belong to exist.

3. External Interface Requirements

3.1 User Interfaces

UI-1: The system's user interface shall conform to modern responsive web design principles, ensuring compatibility across desktop, tablet, and mobile devices.

UI-2: The system shall provide consistent navigation elements including:

- Top navigation bar with search, notifications, inbox, and user profile access
- Sidebar for community navigation and recommendations
- Bottom navigation for mobile devices

UI-3: All interactive elements shall provide visual feedback (hover states, active states, loading indicators) within 100ms of user interaction.

UI-4: The system shall support both light and dark theme modes, allowing users to switch between them in User Settings.

UI-5: The system shall provide contextual help tooltips for complex features such as post creation, community creation and moderation tools, privacy settings, and search filters.

UI-6: All forms shall include input validation with real-time error messages, clear labeling and placeholder text, required field indicators, and submit confirmation dialogs for destructive actions.

UI-7: The system shall support keyboard navigation and shortcuts for navigating between posts (J/K keys), upvote/downvote (A/Z keys), opening post details (Enter key), and search activation (/ key).

UI-8: Accessibility requirements:

- WCAG 2.1 Level AA compliance
- Screen reader compatibility
- Sufficient color contrast ratios (minimum 4.5:1)
- Alternative text for images
- Keyboard-only navigation support

UI-9: The post detail page shall display threaded comments with collapsible comment threads, visual indentation for nested replies (maximum 10 levels), sort options (best, new, top, controversial), and inline reply and edit forms.

UI-10: The system shall display loading states and skeleton screens during data fetching to improve perceived performance.

3.2 Software Interfaces

SI-1: Database Management System

SI-1.1: The system shall interface with a relational database (PostgreSQL/MySQL) or NoSQL database (MongoDB) for persistent data storage.

SI-1.2: Database operations shall use prepared statements or ORM (Object-Relational Mapping) to prevent SQL injection attacks.

SI-1.3: The system shall maintain the following data tables/collections: Users, Communities, Posts, Comments, Notifications, Messages, and Reports.

SI-2: Authentication Service

SI-2.1: The system shall integrate with an authentication service (e.g., JWT, OAuth 2.0, Firebase Auth) for user registration and login, session management, password reset functionality, and token generation and validation.

SI-2.2: The authentication service shall return user credentials and access tokens that expire after a configurable time period (default: 24 hours).

SI-2.3: The system may support social authentication providers: Google OAuth, GitHub OAuth, and Facebook Login.

SI-3: File Storage Service

SI-3.1: The system shall interface with a file storage service (e.g., AWS S3, Cloudinary, local storage) for user avatars, community banners, post images and albums, and video uploads.

SI-3.2: The storage service shall support image format validation (JPEG, PNG, GIF, WebP), video format validation (MP4, WebM), maximum file size limits (images: 10MB, videos: 100MB), and automatic image optimization and thumbnail generation.

SI-3.3: The system shall generate signed URLs for secure file access with expiration times.

SI-4: Email Service

SI-4.1: The system shall integrate with an email service provider (e.g., SendGrid, AWS SES, Mailgun) for account verification emails, password reset emails, notification digest emails, and moderation alerts.

SI-4.2: All emails shall include unsubscribe links (for notification emails), plain text and HTML versions, and sender domain authentication (SPF, DKIM).

SI-5: Search Engine (Optional Enhancement)

SI-5.1: The system may integrate with a search engine service (e.g., Elasticsearch, Algolia) for full-text search across posts, comments, communities, and users with advanced filtering and faceted search.

SI-5.2: Search indexing shall update within 5 seconds of content creation or modification.

SI-6: Content Moderation API (Optional)

SI-6.1: The system may integrate with content moderation services (e.g., AWS Rekognition, Google Cloud Vision API) to detect inappropriate images, filter spam content, and identify potentially harmful text.

SI-7: Video Processing Service (Optional)

SI-7.1: For video posts, the system may integrate with a video processing service (e.g., FFmpeg, AWS Elemental MediaConvert) to transcode videos to web-compatible formats, generate video thumbnails, and create multiple quality versions (360p, 720p, 1080p).

SI-8: Real-time Communication Service

SI-8.1: The system shall use WebSocket protocol or Server-Sent Events (SSE) for real-time notifications, live comment updates on post detail pages, and instant messaging in DMs and group chats.

SI-8.2: The system shall fall back to HTTP polling if WebSocket connections fail.

SI-9: Analytics Service (Optional)

SI-9.1: The system may integrate with analytics platforms (e.g., Google Analytics, Mixpanel) to track page views and user engagement, popular communities and posts, user retention metrics, and feature usage statistics.

3.3 Hardware Interfaces

HI-1: No direct hardware interfaces are required for this web-based application.

HI-2: The system shall operate on standard web server hardware meeting minimum specifications: Multi-core processor (minimum 2 cores), Minimum 4GB RAM (8GB recommended for production), and Minimum 50GB storage (expandable based on user-generated content).

3.4 Communications Interfaces

CI-1: HTTP/HTTPS Protocol

CI-1.1: All client-server communications shall use HTTPS (TLS 1.2 or higher) to ensure data encryption in transit.

CI-1.2: The system shall enforce HTTPS by redirecting all HTTP requests to HTTPS endpoints.

CI-1.3: API endpoints shall follow RESTful design principles using GET for data retrieval, POST for resource creation, PUT/PATCH for resource updates, and DELETE for resource deletion.

CI-1.4: API URL structure shall conform to the standard format:

`https://<hostname>/api/<resource>/<id?>?query=xxx`

CI-1.5: API URL conventions:

- All user-facing endpoints shall begin with `/<hostname>/api/`
- All admin-facing endpoints shall begin with `/<hostname>/api/admin/`
- Resource names shall always be plural (e.g., `/api/users`, `/api/posts`, `/api/communities`)
- GET requests shall NOT use request body; parameters shall be passed via query strings
- Resource IDs shall be included in the path when accessing specific resources
- Query parameters shall NOT be embedded in the path

CI-1.6: Request and response body format: All field names in JSON request and response bodies shall use snake_case notation (e.g., `user_id`, `created_at`, `post_title`, `community_name`).

CI-1.7: The system shall return appropriate HTTP status codes: 200 (OK), 201 (Created), 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 429 (Too Many Requests), 500 (Internal Server Error).

CI-2: Data Exchange Format

CI-2.1: All API communications shall use JSON (JavaScript Object Notation) as the primary data exchange format.

CI-2.2: All field names in request and response bodies shall use snake_case naming convention.

CI-2.3: API responses shall include standard metadata structure with status, data, message, and timestamp fields.

CI-2.4: Error responses shall include status, error_code, message, errors array with field-specific messages, and timestamp.

CI-2.5: Pagination responses shall include metadata with current_page, total_pages, page_size, and total_items.

CI-3: WebSocket Communication

CI-3.1: Real-time features shall use WebSocket connections over WSS (WebSocket Secure) protocol.

CI-3.2: WebSocket messages shall be formatted as JSON and include message types: notification (new notifications), message (new DM or group chat messages), comment_update (new comments on subscribed posts), and presence (user online/offline status).

CI-4: Email Notifications

CI-4.1: The system shall send email notifications for account verification (required), password reset requests (required), comment replies, post mentions, direct messages, and moderation actions (for moderators).

CI-4.2: Email notification preferences shall be configurable per user in User Settings.

CI-4.3: Users shall be able to unsubscribe from non-essential emails via unsubscribe links.

CI-5: Browser Push Notifications (Optional)

CI-5.1: The system may support browser push notifications using the Web Push API for new notifications, new messages, and mention alerts.

CI-5.2: Users must explicitly grant permission for push notifications.

CI-5.3: Push notifications shall include title, message preview, action buttons (Mark as read, View), and clickable link to relevant content.

CI-6: Rate Limiting

CI-6.1: API endpoints shall implement rate limiting to prevent abuse:

- Authentication endpoints: 5 attempts per 15 minutes per IP
- Post creation: 10 posts per hour per user
- Comment creation: 50 comments per hour per user
- Vote actions: 100 votes per minute per user
- Search requests: 30 requests per minute per user

CI-6.2: Rate limit responses shall return HTTP 429 (Too Many Requests) with retry-after headers.

CI-7: API Versioning

CI-7.1: The API shall include version numbers in the URL path (e.g., /api/v1/posts).

CI-7.2: The system shall maintain backward compatibility for at least one major version.

CI-8: Cross-Origin Resource Sharing (CORS)

CI-8.1: The server shall implement CORS headers to allow controlled access from web browsers.

CI-8.2: CORS shall be configured to allow only trusted origins (the application's own domain).

CI-9: Content Delivery

CI-9.1: Static assets (images, videos, CSS, JavaScript) should be served through a Content Delivery Network (CDN) for improved performance.

CI-9.2: Media files shall be served with appropriate cache headers (max-age: 31536000 for immutable content).